

Single-Node 4×A100 Parallelism Experiment Report

Scaling and Multi-Dimensional Parallelism with Megatron-LM

1. Executive Summary

This report extends the prior 2×A100 DP/TP/PP lab to a single 4×A100-SXM4-40GB node. Question A reuses the original ~1.3B GPT config to measure pure DP scaling 1→2→4. Question B introduces a larger model that requires model parallelism, then compares TP, PP, and 2D combinations (TP×DP, PP×DP, TP×PP, TP4, PP4).

- Original-model DP4 strong scaling: 60646 tok/s (3.77× vs 1GPU, efficiency 94.3%).
- DP4 weak-scaling efficiency (t1/t4): 98.0% with GBS 64→256.
- Large-model 1GPU capacity: OOM (DP alone does not reduce replica memory).
- Highest large-model aggregate throughput: large_PP2DP2_weak at 37486 tok/s.

2. Hardware and Software Setup

Four NVIDIA A100-SXM4-40GB GPUs on one node. Topology matrix shows **NV12** between every GPU pair (full NVLink clique). GPUs 0–1 share NUMA 3; GPUs 2–3 share NUMA 1 — still NVLink-connected, so collectives stay on NVLink rather than traversing SYS for GPU↔GPU traffic.

Software: PyTorch 2.10 (NGC), Megatron-LM be85fc5df, BF16, --mock-data, FlashAttention enabled. Exact dumps: logs/4gpu/00_system_check.log.

3. What Changed from the 2-GPU Experiment

The 2-GPU artifacts (report.pdf, CSVs, logs, profiles) are preserved untouched. This generation writes only under logs/4gpu/, results/4gpu/, profiles/4gpu/, and reports/4gpu/. New work: DP4 scaling on the original model; a larger model for capacity/model-parallel study; multi-dimensional layouts TP2×DP2, PP2×DP2, TP4, PP4, TP2×PP2; expanded Nsight set.

4. 4-GPU NCCL Baseline

AllReduce / AllGather / ReduceScatter from 8 MiB→1 GiB on 4 GPUs. Bus bandwidth must not be read as “twice the 2-GPU number”: increasing N changes both communication volume and collective scheduling.

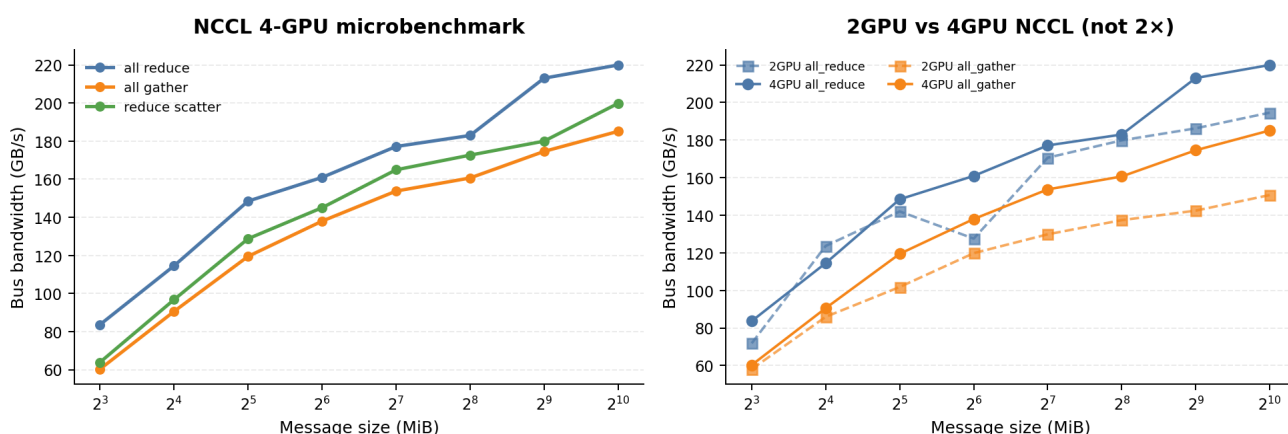


Figure: 4-GPU NCCL busbw and 2GPU vs 4GPU comparison.

At 1 GiB AllReduce: time=7322.2 μs, algbw=146.6 GB/s, busbw=220.0 GB/s (CSV: results/4gpu/nccl_results_4gpu.csv).

5. Original Model: DP Scaling 1 → 2 → 4

Frozen original config: layers=24, hidden=2048, ffn=8192, heads=16, seq=2048, vocab=32000, BF16, micro_batch=1, global_batch=64 (strong). Warmup=20, measured=100.

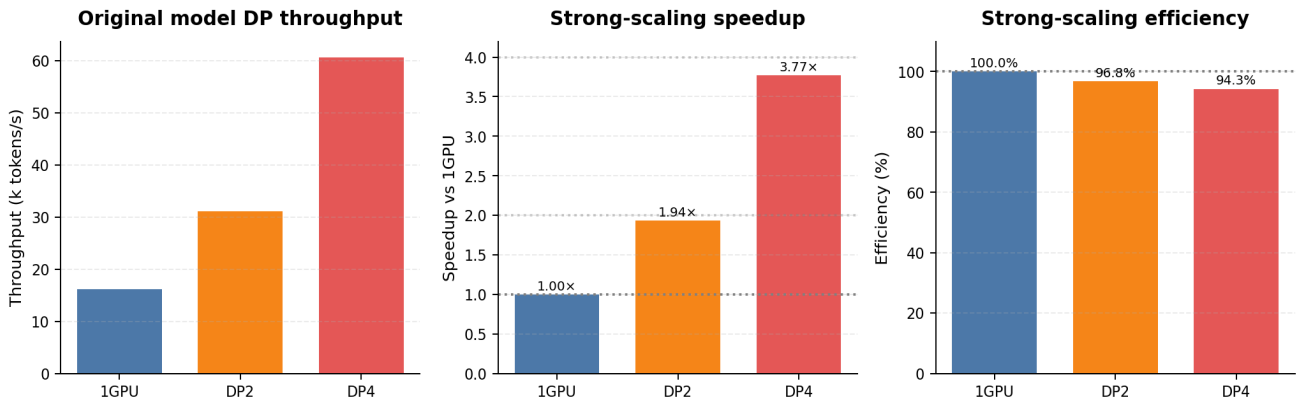


Figure: DP strong scaling throughput, speedup, efficiency.

Config	GPUs	Tok/s	Speedup	Eff%	step_ms	Tok/s/GPU	peak_MB
orig_1GPU	1	16084	1.00	100.0	8149.4	16084	34682
orig_DP2	2	31134	1.94	96.8	4209.9	15567	34682
orig_DP4	4	60646	3.77	94.3	2161.3	15162	34682

5.1 Weak scaling

Weak scaling keeps work/GPU $\approx$ constant: GBS=64,128,256 for DP=1,2,4. Efficiency defined as t_{1GPU} / t_{DP} (ideal ≈ 1.0 when step time stays flat).

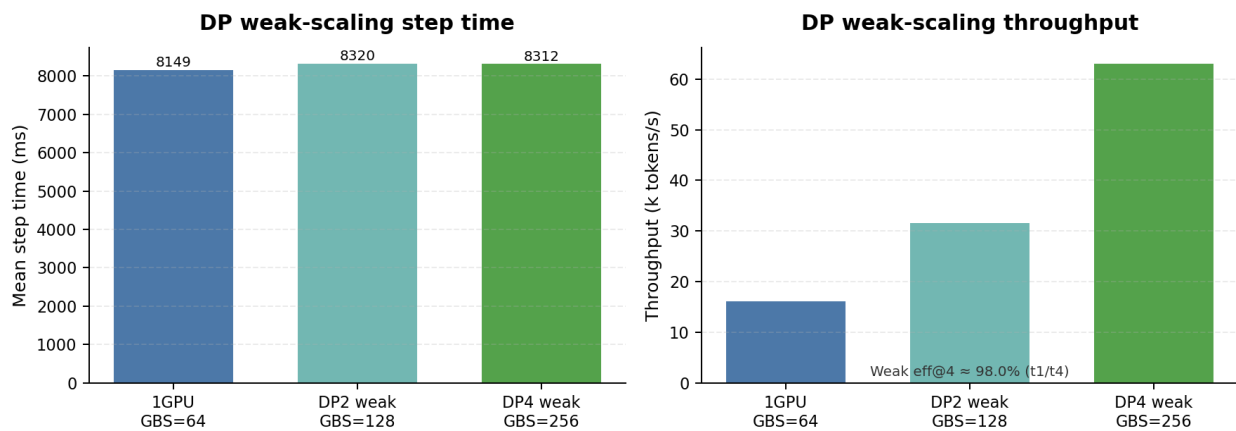


Figure: DP weak-scaling step time and throughput.

6. Large Model Configuration

```
Frozen at Wed Sep 9 10:35:00 UTC 2026
Source: plan_4gpu.txt $6 (adjusted only if capacity tests require)
approx_params=3216506880 (3.217B)
layers=20 hidden=3072 ffn=12288 heads=24 vocab=32000
# Frozen large-model config for 4-GPU experiment
export LARGE_NUM_LAYERS=20
export LARGE_HIDDEN_SIZE=3072
export LARGE_FFN_HIDDEN_SIZE=12288
export LARGE_NUM_ATTENTION_HEADS=24
export LARGE_SEQ_LENGTH=2048
export LARGE_VOCAB_SIZE=32000
export LARGE_GLOBAL_BATCH_SIZE=64
```

Design goal: 1GPU (and therefore pure DP) should OOM or sit beyond a safe margin; TP2/PP2 must fit. Any size adjustment is documented in the config file and frozen before comparable large-model runs.

7. Capacity Results

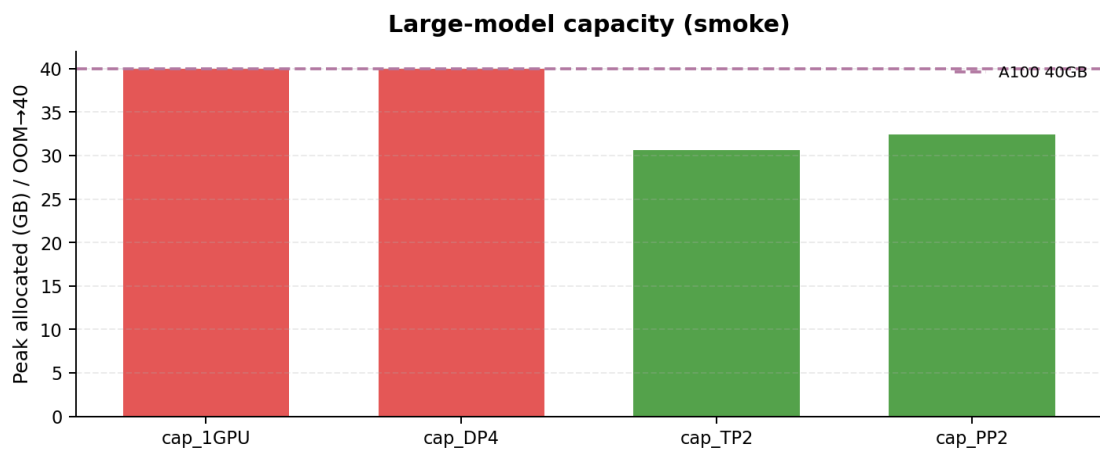


Figure: capacity smoke outcomes (red=OOM).

Config	GPUs	TP	PP	DP	Status	peak_MB
cap_1GPU	1	1	1	1	OOM	—
cap_DP4	4	1	1	4	OOM	—
cap_TP2	2	2	1	1	FITS	31344
cap_PP2	2	1	2	1	FITS	33172

Pure DP4 does not fix single-replica OOM: each rank still holds a full model copy. TP shards weights/activations within a layer; PP shards layers across stages.

8. TP2 and TP2×DP2

TP2 baseline on 2 GPUs; TP2×DP2 places two TP replicas on 4 GPUs. Strong: same GBS as TP2. Weak: 2× GBS. Weak efficiency = $\text{throughput_TP2DP2} / (2 \times \text{throughput_TP2})$.

large_TP2: 17161 tok/s, step=7637.7 ms, peak_alloc=31342.55 MB, tok/s/GPU=8581.

large_TP2DP2_strong: 33471 tok/s, step=3916.0 ms, peak_alloc=31342.54 MB, tok/s/GPU=8368.

large_TP2DP2_weak: 34239 tok/s, step=7656.3 ms, peak_alloc=31342.55 MB, tok/s/GPU=8560.

TP2×DP2 weak-scaling efficiency: 99.8%.

9. PP2 and PP2×DP2

large_PP2: 18992 tok/s, step=6901.4 ms, peak_alloc=33172.09 MB.

large_PP2DP2_strong: 36508 tok/s, step=3590.2 ms, peak_alloc=33172.09 MB.

large_PP2DP2_weak: 37486 tok/s, step=6993.1 ms, peak_alloc=33172.09 MB.

PP2×DP2 weak-scaling efficiency: 98.7%.

Head-to-head weak aggregate: PP2×DP2 wins (37486 vs 34239 tok/s).

10. TP2 vs TP4

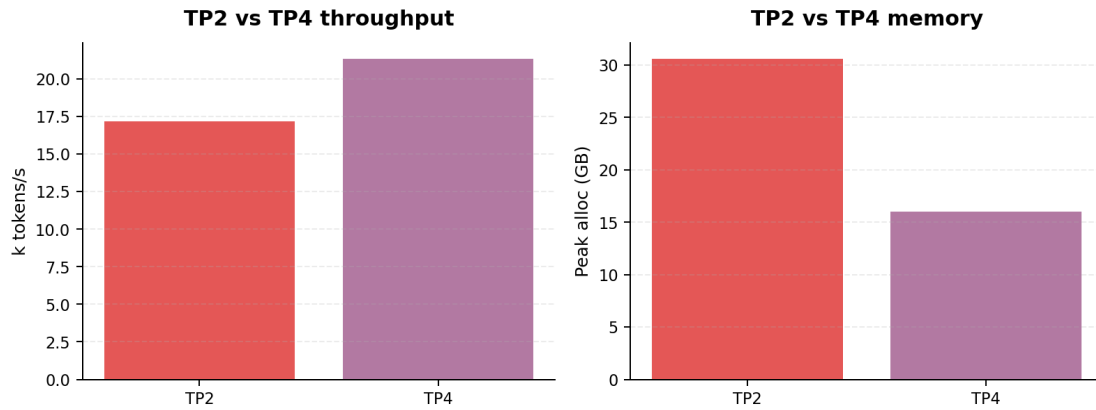


Figure: TP degree scaling.

TP4 vs TP2 throughput ratio=1.24; memory TP2=30.6 GB → TP4=16.0 GB. Higher TP usually raises NCCL share and lowers tok/s/GPU.

11. PP2 vs PP4 (microbatch sweep)

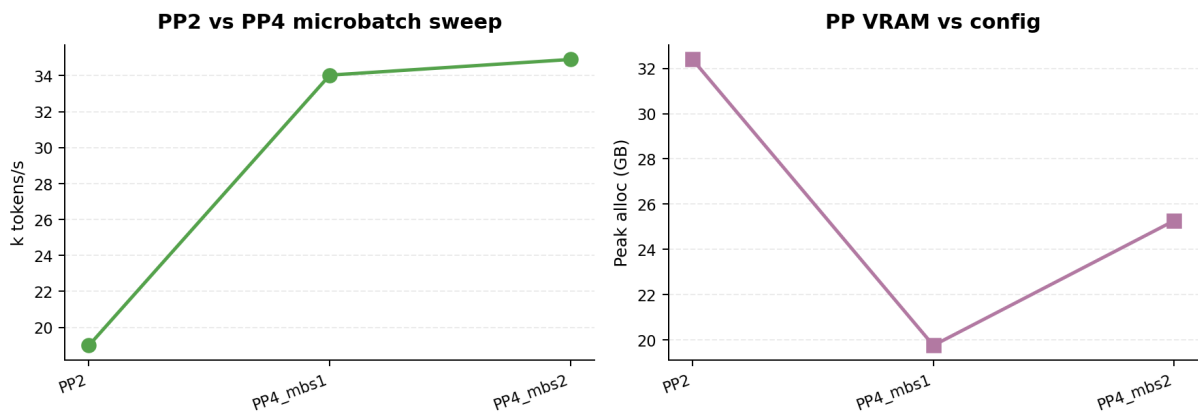


Figure: pipeline depth and microbatch vs throughput/VRAM.

Deeper PP increases bubble fraction for a fixed number of microbatches; raising micro_batch_size fills the pipeline and can improve GEMM efficiency, at the cost of activation memory. Fair comparisons keep micro_batch=1; sweeps are sensitivity, not pure scaling.

12. TP2×PP2

TP2×PP2 on 4 GPUs: 30840 tok/s, step=4250.1 ms, peak_alloc=17262.22 MB. Combines intra-layer TP collectives with inter-stage P2P.

Large-model overview

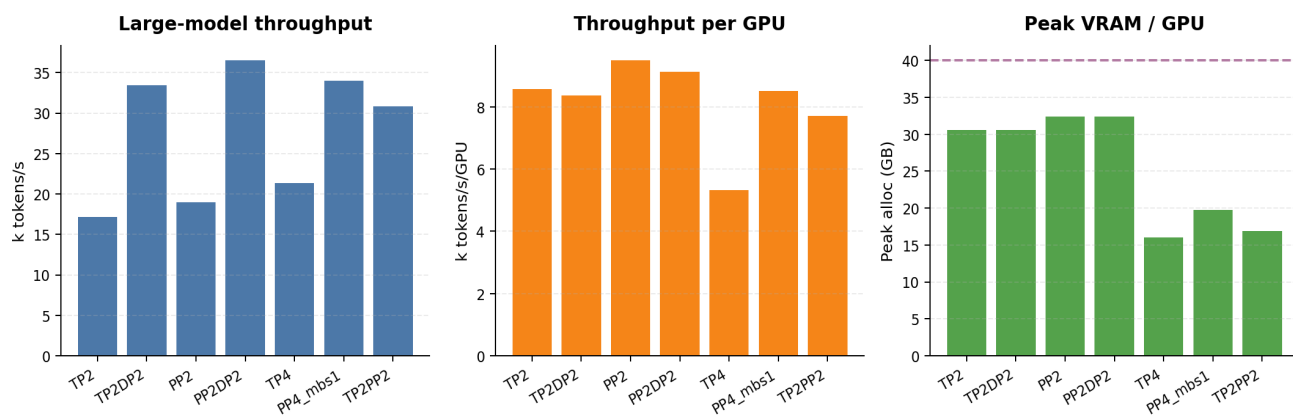


Figure: large-model throughput, per-GPU throughput, VRAM.

13. Nsight Systems Communication Analysis

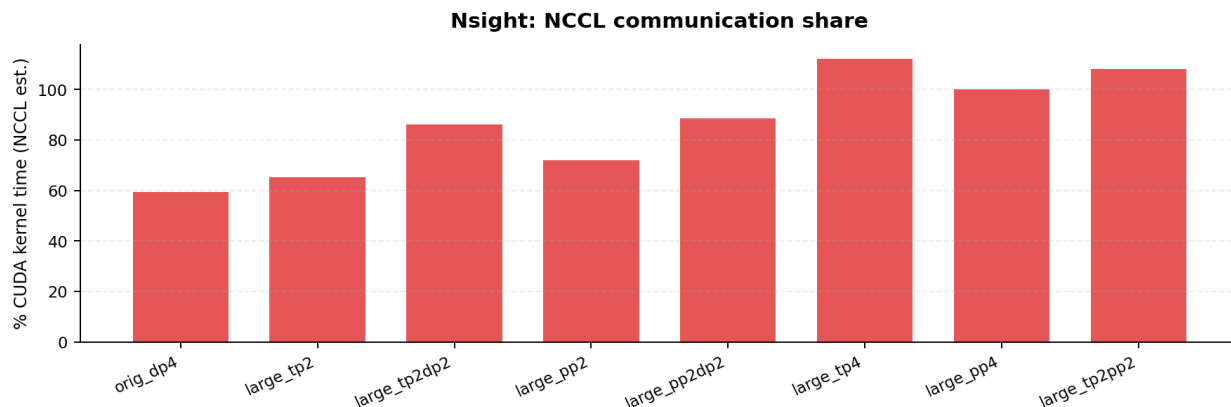


Figure: estimated NCCL share of CUDA kernel time.

Profiles are short steady-state windows (TRAIN_ITERS=30, nsys duration≈50s). DP is dominated by infrequent gradient AllReduce; TP shows frequent bf16 AllReduce/AllGather/ReduceScatter inside layers; PP emphasizes SendRecv with compute bubbles; hybrids mix both patterns. Raw: profiles/4gpu/*.nsys-rep and *_stats.txt.

14. Overall Throughput / Memory / Communication Tradeoff

On this NVLink clique: prefer pure DP while the replica fits; introduce TP or PP only for capacity; use remaining GPUs for DP (TP×DP / PP×DP) rather than maximizing TP degree alone; treat PP microbatch as a tuning knob separate from fair scaling claims.

15. Answers to Study Questions

1–2. DP 1→2→4 / DP4 efficiency: 1GPU=16084, DP2=31134, DP4=60646 tok/s; speedup₄=3.77×, efficiency=94.3%.

3. DP4 weak efficiency: t₁/t₄ = 98.0%.

4–5. Large 1GPU / DP4 fit?: 1GPU=OOM, DP4=OOM — DP does not shrink replica memory.

6–7. TP2/PP2 memory: TP2=FITS, PP2=FITS (peak MB in capacity CSV).

8. TP2×DP2 vs TP2: weak eff=99.8%.

9. PP2×DP2 vs PP2: weak eff=98.7%.

10. TP2×DP2 vs PP2×DP2: PP2×DP2 higher aggregate weak throughput.

11–12. TP2→TP4: throughput ratio=1.24; see Nsight CSV for NCCL share.

13–14. PP bubbles / microbatch: PP4 needs more microbatches to hide bubbles; sweep shows throughput/VRAM tradeoff (fig6).

15. TP2×PP2 Nsight: mixed AllReduce + SendRecv; see nsys_large_tp2pp2_stats.txt.

16–18. Best tok/s / mem / tok/s/GPU: max tok/s=large_PP2DP2_weak (37486); min mem=large_TP4 (16396.83 MB); max tok/s/GPU=large_PP2 (9496).

19. Most frequent communication: Pure TP (and TP4) — per-layer collectives; see Nsight NCCL %.

20. Practical rule: Fit with DP if possible; else minimal TP/PP for capacity, then DP on remaining GPUs; avoid over-sharding TP; tune PP microbatch separately.

16. Conclusions and Practical Guidance

DP scales nearly linearly on this 4-GPU NV12 node for the original model. For models that exceed 40 GB per replica, shard with TP or PP first, then scale out with DP. Avoid TP4 unless memory demands it—communication

share rises quickly. Prefer PP microbatch sweeps after choosing the parallel layout.

17. Reproducibility

```
bash scripts/setup_third_party.sh
bash run_all_4gpu.sh
# or stepwise scripts under scripts/4gpu/
python3 analysis/4gpu/summarize_4gpu.py
python3 analysis/4gpu/generate_report_4gpu.py
# outputs: results/4gpu/*.csv, reports/4gpu/report_4gpu.pdf, report_4gpu.pdf
```

All numeric claims trace to logs/4gpu/*.log, results/4gpu/*.csv, and profiles/4gpu/*_stats.txt. Legacy 2-GPU report.pdf is unchanged.